

Project Report: Hangman Game

Abdullah Mansoor
Basil Uddin
Daanial Tejani

Contents

1	Introduction	2
2	Objectives	2
3	Features	2
3.1	Categories	2
3.2	Gameplay Mechanics	2
3.3	Scoring and Persistence	3
3.4	User-Friendly Design	3
4	Implementation Details	3
4.1	Programming Language	3
4.2	Word Management	3
4.3	Functions	3
4.4	Error Handling	4
4.5	User Interface	4
5	Testing	4
5.1	Functional Testing	4
5.2	Cross-Platform Testing	4
6	Challenges	4
7	Future Enhancements	5
8	Conclusion	5

1 Introduction

The Hangman Game is an interactive command-line application developed in C programming language. It provides an engaging and educational gaming experience where players guess words belonging to different categories, aiming to complete the word within a limited number of incorrect attempts.

2 Objectives

The primary objectives of the project include:

- Providing an entertaining and user-friendly word-guessing game.
- Introducing basic themes for players to choose from.
- Implementing scoring, hints, and record-keeping functionality for a personalized gaming experience.

3 Features

3.1 Categories

Players can select from predefined categories, including:

- Programming
- Sports
- Animals
- Physics
- Maths
- Entire Dictionary (custom word list from a file)

3.2 Gameplay Mechanics

- **Word Guessing:** Players guess letters to reveal a hidden word.
- **Attempts:** Players have five attempts to guess incorrectly before the game ends.
- **Hints:** Players are offered a hint when only one attempt is left, ensuring fairness and engagement.
- **Repeated Letters:** The game restricts repeated letter guesses, warning the player and limiting the number of repetitions.

3.3 Scoring and Persistence

- **Scoring:** Correct guesses and successful completions increase the player's score.
- **Game Records:** Results and scores are saved in a text file (`game_records.txt`) for reference.

3.4 User-Friendly Design

- Clear prompts and color-coded text for ease of use.
- Intuitive input and feedback mechanisms.
- Cross-platform compatibility using conditional compilation for handling terminal I/O and screen clearing.

4 Implementation Details

4.1 Programming Language

The game is implemented in **C**, leveraging libraries such as `stdio.h`, `stdlib.h`, and `string.h`. Platform-specific headers like `conio.h` (Windows) and `termios.h` (Linux) ensure compatibility.

4.2 Word Management

- **Predefined Words:** Words are organized into arrays for each category.
- **Custom Dictionary:** A file-based dictionary allows dynamic word selection.
- **Random Selection:** Randomization using the `rand()` function ensures variety in gameplay.

4.3 Functions

- **Game Core:**
 - `hangman_game()`: Main function managing gameplay logic.
 - `pick_random_word()`: Selects a random word based on category.
 - `initialize_display()`: Initializes the masked word display.
- **Utilities:**
 - `show_hangman()`: Displays hangman graphics based on remaining attempts.
 - `hint()`: Provides a hint by revealing one letter in the word.
 - `saveGameRecords()`: Saves game results to a file.
 - `showGameRecord()`: Displays previously saved game records.
- **Input Handling:**
 - `is_repeated()`: Checks if a guessed letter has already been used.

4.4 Error Handling

- File operations handle errors gracefully with appropriate messages.
- Input validation ensures robustness against invalid entries.

4.5 User Interface

Color-coded text highlights:

- Correct and incorrect guesses.
- Remaining attempts.
- Themes and prompts.

5 Testing

5.1 Functional Testing

The following scenarios were tested:

1. Correct guesses.
2. Incorrect guesses reducing attempts.
3. Validating repeated letters.
4. Hint functionality.
5. Handling edge cases (e.g., empty or invalid file).

5.2 Cross-Platform Testing

The game was tested on:

- Windows using `cmd`.
- Linux using terminal.

6 Challenges

- Implementing platform-independent terminal I/O.
- Ensuring robust handling of edge cases (e.g., dictionary file errors).
- Maintaining user experience across diverse setups.

7 Future Enhancements

- **Graphical Interface:** Introducing a GUI for enhanced interaction.
- **Multiplayer Mode:** Allowing multiple players to compete.
- **Advanced Categories:** Adding more themes and dynamically fetching words from online sources.
- **Adaptive Difficulty:** Adjusting difficulty based on player's performance.

8 Conclusion

The Hangman Game project successfully combines entertainment and learning with intuitive design and cross-platform functionality. By leveraging structured programming, the project provides a reliable and engaging user experience. Future enhancements can further expand its scope and usability.